

הסקה ואימון של מודלי שפה גדולים מקומית

Local Inference and Training of Large Language Models

On-Premises LLM Deployment: Utilizing Ollama, LoRA, and AirLLM (L08)

ד"ר יורם סגל

כל הזכויות שמורות - © ד"ר יורם סגל

June 2026

סיכום שיעור L08

הסקה ואימון של מודלי שפה גדולים מקומית

מרצה: ד"ר יורם סגל

מילות מפתח: הסקה (Inference), אימון (Training), GPU מול CPU, CUDA, PTX, SASS, שלב Prefill, שלב Decode, זיכרון VRAM, קוונטיזציה (Quantization), Hugging Face, Ollama, פורמט SafeTensors, פורמט GGUF, LoRA, AirLLM, PEFT, OLoRA, QLoRA, זיכרון וירטואלי (Virtual Memory), Paging, רישוי מודלים.

סיכום זה מתאר את המסלול המלא של עבודה מקומית עם מודלי שפה גדולים: מהחומרה (GPU/CPU), דרך פורמטים, רישוי וקוונטיזציה, ועד לאימון יעיל באמצעות LoRA והרצת מודלים ענקיים על CPU בעזרת AirLLM ועקרונות הזיכרון הווירטואלי ממערכות הפעלה.

כל הזכויות שמורות לד"ר יורם סגל ©

י"פ

תוכן העניינים

1	מבוא: שלוש דרכים להריץ מודלים	1
2	שלוש הגישות להרצת מודלים	1.1
2	עלות ואבטחת מידע בעבודה מקומית	1.2
3	חומרה: CPU מול GPU	2
3	מקביליות מול עיבוד סדרתי	2.1
3	CUDA, PTX ו-SASS	2.2
4	הרחבה – Fat Binary והידור בזמן אמת (JIT)	2.3
5	ידע מתקדם – התפתחות ה-Warp Divergence: מ-Pascal ל-Volta	2.4
7	הסקה (Inference): Decode ו-Prefill	3
8	שני שלבי ההסקה	3.1
8	ייצוגי ביניים (Latent)	3.2
9	VRAM – זיכרון הכרטיס הגרפי	3.3
9	הרחבה – הבסיס המתמטי ומטמון ה-KV Cache	3.4
9	ידע מתקדם – הסקה מבוזרת (Disaggregated Serving)	3.5
11	עבודה עם Hugging Face: בחירת מודל	4
11	שלבי בחירת מודל	4.1
12	רישוי מודלים (License)	4.2
12	פורמטים: GGUF ו-SafeTensors	4.3
13	הרחבה – מדוע SafeTensors בטוח: pickle מול Flat Buffer	4.4

15	קוונטיזציה (Quantization)	5
16	ידע מתקדם – QLoRA ו-NormalFloat (NF4)	5.1
17	Ollama: הרצה מקומית	6
19	אימון ו-LoRA	7
19	אימון מול הסקה	7.1
19	Transfer Learning ו-PEFT	7.2
20	הרעיון של LoRA	7.3
21	OLoRA ו-QLoRA	7.4
21	הרחבה – הבסיס המתמטי של LoRA ו-OLoRA	7.5
23	AirLLM: הרצת מודלים ענקיים על CPU	8
23	הרעיון	8.1
23	תזכורת: זיכרון וירטואלי ממערכות הפעלה	8.2
24	המחיר: Latency מול זיכרון	8.3
24	הרחבה – כיצד AirLLM מנצל את mmap	8.4
24	ידע מתקדם – חזית המחקר: FlexGen, LLM in a Flash	8.5
24	ו-PagedAttention	
27	המשימה המעשית	9
27	מה צריך לעשות	9.1
28	סיכום המסע	9.2

1 מבוא: שלוש דרכים להריץ מודלים

בשיעור זה אנו עוברים את המסלול המלא של עבודה מקומית עם סוכני בינה מלאכותית במחשבים שלנו. ראשית נדבר על החומרה ועל ההבדל בין CPU ל-GPU ועל האופן שבו הטרנספורמר בא לידי ביטוי בחומרה. לאחר מכן נראה כיצד מורידים מודלים באמצעות Ollama ו-Hugging Face, ונתמודד עם הבעיה שמודלים גדולים קשים מאוד להרצה. לבסוף נציג שתי שיטות מרכזיות: LoRA לאימון יעיל של מודלי ענק, ו-AirLLM להרצת מודלים גדולים על CPU רגיל.

1.1 שלוש הגישות להרצת מודלים

קיימות שלוש אפשרויות עיקריות להרצת מודלים:

1. **API** – הגישה הבסיסית והפשוטה ביותר. פונים לשירות חיצוני כגון OpenAI או Claude, משלמים לפי טוקנים ועובדים. זו תמיד נקודת ההתחלה הטובה לפרויקט חדש: עושים ניסוי, ואם מתאים – אפשר להמשיך.
2. **ענן (Cloud GPU)** – רוכשים כרטיסי GPU ושרתים בענן ומפעילים את המחשב מרחוק. היתרון: אין צורך לקנות חומרה, משלמים לפי שימוש; החיסרון: יקר יותר לאורך זמן.
3. **מקומי (On-Premise)** – מריצים את כל המודלים על המחשב שלנו. ככל שהמחשב חזק יותר, החיים קלים יותר, אך קיימת עלות של כוח חישוב.

טבלה 1: השוואת שלוש גישות ההרצה

גישה	יתרונות	חסרונות
API	פשטות מרבית, ללא חומרה, התחלה מהירה	תשלום לפי טוקנים, המידע יוצא החוצה
ענן GPU	אין רכישת חומרה, תשלום לפי שימוש	יקר יותר, המידע עדיין בענן
מקומי On-Premise	פרטיות מלאה, ללא תשלום שוטף	השקעת הון ראשונית, עלות חשמל

1.2 עלות ואבטחת מידע בעבודה מקומית

בעבודה מקומית (On-Premise) העלות עוברת להשקעה הונית: במקום לשלם לפי שימוש, קונים מראש מחשב חזק (לעיתים בעלות של עשרות אלפי דולרים) ומקווים שההשקעה תחזיר את עצמה. היתרון העצום שלעיתים אין לו מחיר הוא **אבטחת המידע**: שום דבר אינו יוצא מהארגון. במערכות ביטחוניות, בצה"ל ובמקומות רגישים זהו שיקול מכריע, משום שכאשר מוציאים מידע לענן לעולם אי אפשר לדעת בוודאות מי מאזין באמצע ומה נעשה במידע.

2 חומרה: CPU מול GPU

2.1 מקביליות מול עיבוד סדרתי

ל-GPU יש אלפי ליבות (cores) הפועלות במקביל – multithreading מסיבי של אלפי threads. ל-CPU יש מעט ליבות, אך הוא יעיל יותר כאשר המשימה סדרתית ומורכבת, כלומר כשפעולה אחת תלויה בתוצאת הפעולה הקודמת. ל-GPU אין יתרון במשימות סדרתיות.

במודלים של טרנספורמרים מתבצעות המון פעולות של הכפלת מטריצות בתבנית קבועה. כאשר התבנית קבועה (למשל $A + B$ כאשר רק הערכים משתנים אך הפעולה זהה) וחוזרת על עצמה פעמים רבות, ה-GPU מהיר בסדרי גודל מ-CPU, ולכן הוא הפתרון האולטימטיבי למודלי שפה גדולים.

2.2 SASS, PTX, CUDA ו-SASS

CUDA היא פלטפורמת התכנות של NVIDIA. כותבים בה בשפה מורחבת המבוססת על C++, עם ספריות מיוחדות. שרשרת ההידור (compilation) עוברת שלושה שלבים:

1. CUDA – קוד המקור בשפה מורחבת.
2. PTX – שפת אסמבלר אוניברסלית של ה-GPU, שאינה תלויה בכרטיס מסוים ומתאימה לכל כרטיסי ה-GPU.
3. SASS – השלב שבו הקוד מהודר באופן ייעודי (dedicated) לחומרה הספציפית של כרטיס המסך, אחד לאחד.

Warp Divergence: כאשר תוך כדי עבודה מקבילית מופיעה הסתעפות (branch) – חלק מהפעולות הולכות לכיוון אחד וחלק לכיוון אחר – העיבוד הופך מסדרתי ואובד יתרון המקביליות. לכן מתכננים מראש שכל קבוצת פעולות זהות תהיה יחד.

Cache: ההידור מתבצע בכל הרצה מחדש (כמו אינטרפרטר), והדבר שמציל אותנו מהחזרתיות הוא ה-cache של הכרטיס: אם דבר לא השתנה, אין צורך להדר שוב.

2.3 הרחבה – Fat Binary והידור בזמן אמת (JIT)

מהדר ה-NVCC אורז יחדיו את קוד ה-PTX הווירטואלי ואת קוד ה-SASS הייעודי לתוך קובץ משולב המכונה Fat Binary. גישה זו משיגה בו-זמנית אופטימיזציה מרבית (דרך ה-SASS) ותאימות קדימה (Forward Compatibility, דרך ה-PTX). שפת ה-PTX (Parallel Thread Execution) משמשת כסט פקודות וירטואלי (Virtual ISA), בעוד ה-SASS (Streaming Assembly) הוא הקוד הבינארי המותאם אחד-לאחד לכרטיס הספציפי.

כאשר מריצים את התוכנה על כרטיס חדש שעבורו אין SASS תואם בקובץ, מנהל ההתקן (Driver) מבצע הידור בזמן אמת (JIT Compilation) מתוך ה-PTX אל ה-SASS הנדרש. תהליך זה יוצר השהיה ראשונית, ולכן מנהל ההתקן שומר את התוצאה במטמון ייעודי (JIT Cache). כך מוסבר מדוע ההרצה הראשונה של מודל או ספרייה חדשה (למשל בסביבות Triton או PyTorch לאחר עדכון גרסה) עשויה להיות איטית משמעותית מההרצות העוקבות.

2.4 ידע מתקדם – התפתחות ה-Warp Divergence: מ-Pascal ל-Volta

מנגנון ה-Warp Divergence עבר התפתחות ארכיטקטונית חשובה. בארכיטקטורות שקדמו ל-Volta (כגון Pascal), כל התהליכונים ב-Warp חלקו מונה תוכנית (Program Counter) יחיד. לכן החומרה נאלצה להשתמש במסכת פעילות (Active Mask) ולהשבית את התהליכונים שאינם בענף הנוכחי, מה שהוביל לעיבוד סדרתי ולאובדן המקביליות באותו מחזור.

החל מארכיטקטורת Volta ואילך (V100, A100, H100), הוצגה טכנולוגיית תזמון תהליכונים עצמאי (Independent Thread Scheduling). היא מעניקה לכל תהליכון מונה תוכנית ומחסנית קריאות משלו, ומאפשרת התפצלות והתלכדות ברמת תת-Warp, אף שעקרון ה-SIMT (Single Instruction, Multiple Threads) נשמר ברמת הליבה. עם זאת, השאיפה התכנותית נותרת למזער הסתעפויות כדי לנצל את ליבות החישוב במלואן.

3 הסקה (Inference) : Prefill 1-Decode

כאשר מריצים מודל מאומן כדי לקבל תשובה, ההסקה מתבצעת בשני שלבים בעלי אופי שונה לחלוטין.

3.1 שני שלבי ההסקה

שלב ה-Prefill: המודל מקבל את כל הקשר (context window) בבת אחת, מבצע את כל ההכפלות המטריציוניות במקביל על ה-GPU, ומחשב את כל הייצוגים. בסוף השלב מתקבל **טוקן אחד** בלבד. צוואר הבקבוק כאן הוא כוח החישוב של ה-GPU (compute-bound).

שלב ה-Decode: מכניסים את הטוקן חזרה לחלון ההקשר, מקבלים את הטוקן הבא, וחוזר חלילה – טוקן אחרי טוקן. זוהי פעולה **סדרתית** ולכן צוואר בקבוק. מכיוון שב-Prefill כבר חושבו כל המשקלים, בכל טוקן משתנה רק חלק קטן מהם, אך יש להביא בכל פעם את המשקלים מהזיכרון. לכן כאן צוואר הבקבוק הוא **הזיכרון** של הכרטיס (memory-bound).

טבלה 2: השוואה בין Prefill ל-Decode

מאפיין	Prefill	Decode
אופי החישוב	מקבילי	סדרתי
אופי אלגברי	הכפלת מטריצה במטריצה (GEMM)	הכפלת מטריצה בווקטור (GEMV)
פלט	טוקן ראשון + בניית KV Cache	טוקן בודד בכל איטרציה
צוואר הבקבוק	כוח חישוב (compute)	זיכרון הכרטיס (memory)
המגבלה העיקרית	מספר הליבות וה-FLOPs	רוחב הפס וגודל ה-VRAM

3.2 ייצוגי ביניים (Latent)

ייצוגי הביניים הם התוצאות שעוברות בין שכבה לשכבה במודל ה-latent. כל שכבה מבצעת הכפלות ומפיקה ייצוג משלה. הרעיון העדין: אם כבר ידועה התשובה בשכבות מסוימות והיא אינה משתנה בעקבות הטוקן שנוסף, אין צורך לחשב מחדש את כל המסלול – בדומה ל-state machine.

3.3 VRAM – זיכרון הכרטיס הגרפי

המונח VRAM (Video RAM) הוא שם היסטורי שמקורו בכרטיסים הגרפיים. זהו הזיכרון של ה-GPU. בעת רכישת כרטיס GPU זהו אחד הפרמטרים החשובים ביותר: לשלב ה-Prefill חשובים מספר הליבות, אך לשלב ה-Decode מה שקובע הוא גודל ה-VRAM. מומלץ לבדוק את המפרט הטכני של כרטיס המסך במחשב שלכם.

3.4 הרחבה – הבסיס המתמטי ומטמון ה-KV Cache

השוני בין השלבים נובע מהמתמטיקה שביסודם. שלב ה-Prefill מבוסס על הכפלת מטריצה במטריצה (Dense Matrix-Matrix Multiplication – GEMM), פעולה המנצלת ביעילות מרבית את ליבות הטנזור (Tensor Cores) שבכרטיס, ולכן היא חסומה רק בכוח החישוב (FLOPS). תוצר השלב הוא **מטמון המפתח-ערך** (KV Cache) – מצב הביניים (אותם "ייצוגי ביניים" שתוארו לעיל) הנשמר בזיכרון ומשמש להמשך יצירת הטקסט.

שלב ה-Decode, לעומת זאת, דורש הכפלת מטריצת המשקלים בווקטור של הטוקן הבודד (Matrix-Vector). כמות החישובים לכל משקל צונחת, והליבות ממתינות בעוד המשקלים וה-KV Cache ההולך ותופח נשלפים מהזיכרון הפיזי (VRAM/HBM). לכן השלב חסום ברוחב הפס של הזיכרון.

3.5 ידע מתקדם – הסקה מבוזרת (Disaggregated Serving)

ההבחנה בין צווארי הבקבוק הסותרים הובילה לפרדיגמה עדכנית בחזית המחקר – Disaggregated Serving. מחקרים מובילים משנת 2024, כגון Splitwise (ISCA 2024) ו-DistServe (OSDI 2024), הראו ששילוב משימות Prefill ו-Decode על אותו כרטיס יוצר התנגשויות (Interference) הפוגעות בביצועים. הפתרון הוא הפרדה פיזית: קבוצת כרטיסים עתירת כוח חישוב מבצעת אך ורק Pre-fill, ומעבירה את ה-KV Cache דרך רשת מהירה לקבוצת כרטיסים שנייה, עתירת זיכרון ורוחב פס, המבצעת אך ורק Decode. כך נמנעת ההתנגשות ומושגת עלות-תועלת מרבית במרכזי נתונים.

4 עבודה עם Hugging Face: בחירת מודל

Hugging Face הוא המקור המרכזי למודלים – "ה-Youtube של מודלי השפה". כל מי שרוצה להיחשב לאיש מקצוע חייב שיהיה לו חשבון שם. בעת בחירת מודל יש לעבור סדרת שלבים.

4.1 שלבי בחירת מודל

1. **משימה (Task)** – להחליט מה רוצים: מודל שעונה לאנשים, מודל המייצר תמונות, מודל המייצר סאונד וכדומה.
2. **רישיון (License)** – לבדוק את תנאי הרישוי (ראו להלן).
3. **פורמט (Format)** – לוודא שהפורמט מתאים לסביבת העבודה.
4. **גודל המודל** – "שאלת המיליון דולר". מודל גדול אינו בהכרח מתאים למשימה; יש לבדוק אם הוא נכנס ל-VRAM.
5. **תגובתיות (Latency)** – תוך כמה זמן מתקבלת תשובה והאם זה עומד בציפיות.

משל המשאית והאופנוע

מודל גדול אינו בהכרח טוב יותר למשימה שלכם. משאית גדולה יותר מאופנוע, אך אינה בהכרח טובה יותר בתמרון בתנועה. שיקול גודל המודל הוא קריטי.

4.2 רישוי מודלים (License)

הרישוי עלול להפיל ארגון שלם. למודלים ב-Hugging Face יש תנאי רישיון מגוונים: יש רישיונות חופשיים לחלוטין; יש כאלה המתירים שימוש בתנאי שמציינים שהשתמשתם במודל; ויש כאלה המתירים שימוש ללמידה בלבד, ולמטרות מסחריות נדרשת פנייה לבעלים. תמיד מומלץ להיוועץ ביועץ משפטי בארגון רציני.

כלל אצבע לתלות ברישיון

אם תוציאו את הרכיב המורשה והמוצר שלכם יפסיק לתפקד – אין לו חלופה בלעדיו – אתם תלויים בו וחשופים משפטית. אם המוצר ממשיך לעבוד גם בלעדיו (הרכיב אינו מהותי) – החשיפה נמוכה. אין באמור ייעוץ משפטי.

4.3 פורמטים: GGUF ו-SafeTensors

SafeTensors: פורמט שאין בתוכו קוד כלל – אין פקודות קריאה/כתיבה. הוא מכיל רק תיאור של השכבות והארכיטקטורה (מטא-נתונים ב-JSON) ואת רשימת המשקלים. לכן הוא בטוח לטעינה: שום דבר בו אינו קוד בר-הרצה. עדיין קיימות סכנות (למשל מודל שאומן בכוונה זדונית), אך רמת הסיכון יורדת. בארגון, אם נתקלים במודלים שלא הורדו בפורמט SafeTensors – ראוי להרים דגל.

GGUF: פורמט שמרכז משקולות ומטא-נתונים בקובץ דחוס ונוח להרצה מקומית. הקובץ מכיל את המפרט הטכני של המודל ומאפשר שליטה נוחה, ולכן Ollama יודע לקרוא אותו.

4.4 הרחבה – מדוע SafeTensors בטוח: pickle מול Flat Buffer

כדי להבין לעומק את יתרון ה-SafeTensors כדאי להכיר את הסיכון בפורמטים המסורתיים. קבצי PyTorch ישנים (כגון

`pytorch_model.bin`) נשענים על מודול ה-pickle של Python, שנועד לשחזור אובייקטים וקוד – ולכן טעינת מודל נגוע עלולה להריץ קוד שרירותי וזדוני על מכונת היעד בעת פריסת הקובץ לזיכרון.

SafeTensors תוכנן מהיסוד כמבנה נתונים שטוח (Flat Byte Buffer) המכיל מטא-נתונים ב-JSON ומערכי מספרים גולמיים בלבד, ללא יכולת להכיל קוד בר-הרצה. מעבר לביטחון, מבנה זה מאפשר מיפוי זיכרון ישיר (mmap) של הקובץ מהדיסק אל הזיכרון הווירטואלי, ובכך מאיץ דרמטית את זמני טעינת השכבות.

5 קוונטיזציה (Quantization)

משקלי המודל הם ערכים מספריים בייצוג Floating Point. הבעיה היא כמה ספרות לשמור אחרי הנקודה. **קוונטיזציה** היא שיטה לדחוס מידע על חשבון הדיוק: ככל שמורידים את מספר הביטים לכל משקל, חוסכים זיכרון. לדוגמה, מעבר מ-FP32 ל-FP16 מקטין את הזיכרון בחצי (במקום 32 ביט לכל משקל – 16 בלבד). המחיר הוא ירידה אפשרית בדיוק, אך לעיתים זהו "overkill" מיותר, ויש אף מחקרים שמצאו שהקטנה דווקא העניקה למודל יותר גמישות ותוצאות טובות יותר במקרים מסוימים.

טבלה 3: רמות קוונטיזציה אופייניות (ביטים למשקל)

ייצוג	ביטים	הערה
FP32	32	דיוק מלא, צריכת זיכרון מרבית
FP16	16	חצי מהזיכרון, איזון טוב
FP8	8	דחיסה משמעותית
Q4	4	קוונטיזציה אגרסיבית
Q2	2	דחיסה קיצונית לבדיקת "שחרור צנרת"

התחלה עם Q2

בתחילת העבודה עם מודל כדאי להתחיל ב-Q2 רק כדי לוודא שהצנרת עובדת – שהכול זורם והמודל רץ. לא מתייחסים לאיכות התוצאה בשלב זה, אלא רק לכך שהמערכת פועלת.

5.1 ידע מתקדם – QLoRA ו-NF4 NormalFloat

קפיצת מדרגה מחקרית בקוונטיזציה הוצגה בשיטת QLoRA (Dettmers et al., NeurIPS 2023). המחקר הראה שהורדת ביטים גנרית (למשל מ-FP16 לייצוג שלם של 4 ביט) עלולה לפגוע בדיוק בשל פיזור לא-סימטרי של המשקלים. הפתרון הוא סוג נתונים ייעודי, 4-bit NormalFloat (NF4), המותאם סטטיסטית להתפלגות הנורמלית הטבעית של משקלי רשתות נוירונים מאומנות, ומאפשר דחיסה ל-4 ביט כמעט ללא אובדן איכות.

המחקר הוסיף שני מנגנונים משלימים: קוונטיזציה כפולה (Double Quantization), הדוחסת אף את מקדמי הנרמול, ואופטימיזציה מדופדפים (Paged Optimizers), המונעים קריסת זיכרון בעת חריגות רגעיות. שילוב זה הפך לסטנדרט תעשייתי להרצת ואימון מודלי ענק על חומרה ביתית מתקדמת.

6 Ollama: הרצה מקומית

Ollama הוא כמו "נגן ה-Youtube של המודלים": הוא יודע לקרוא קובץ GGUF ואת ה-Modelfile (שבו מתואר מה יש במודל), לטעון את המודל ולהריץ אותו. Ollama חושף API תואם OpenAI, ולכן אפשר לחבר אליו כלים חיצוניים – למשל Claude CLI – על ידי הצבעת ה-API המקומי אל Ollama, ולעבוד עם כל היכולות (סקילים, סשנים) מקומית במחשב.

פקודות Ollama בסיסיות

```
# Pull and run a model locally
ollama pull llama3
ollama run llama3 "What is the capital of Israel?"

# List installed models and serve the local API
ollama list
ollama serve          # exposes an OpenAI-compatible API on
                        localhost:11434

# Point an OpenAI-compatible client to the local server
export OPENAI_BASE_URL="http://localhost:11434/v1"
export OPENAI_API_KEY="ollama"
```

לאחר שמבינים שהמודל רץ מקומית, אפשר להעלות אותו לשרת, לתת כתובת URL ולאפשר לאנשים אחרים להשתמש בו. יש לזכור תמיד את נושא הרישוי של המודלים.

7 אימון ו-LoRA

7.1 אימון מול הסקה

באימון (Training) משנים את המשקולות על בסיס פונקציית Loss, גרדיאנט ירידה (Gradient Descent) ו-Back-propagation. זהו תהליך כבד הדורש זמן ומשאבים. בהסקה (Inference) הכול מבוסס על מה שמכניסים לחלון ההקשר. אימון דורש לא רק משקולות אלא גם גרדיאנטים, אקטיבציות ו-Optimizer (כגון Adam). לכן דרישת ה-VRAM באימון יכולה להיות פי 3 עד פי 5 מאשר בהסקה. זהו ניואנס חשוב שכדאי להכיר.

7.2 PEFT ו-Transfer Learning

לעיתים רוצים לקחת מודל קיים ולהתאים אותו (Transfer Learning) – למשל מודל שיועד לסווג בין סוסים לכלבים, ולהתאים אותו לסיווג בין שולחנות לכיסאות. במקום לאמן מיליארדי פרמטרים, PEFT (Parameter-Efficient Fine-Tuning) מקפיא את רוב המודל ומאמן רק רכיבים קטנים. הגישה שומרת על ידע הבסיס ומצמצמת דרמטית את הזיכרון.

7.3 הרעיון של LoRA

ב-LoRA (Low-Rank Adaptation) ניגשים למטריצת המשקלים W_0 (כל המשקלים של המודל, למשל 70 מיליארד פרמטר), **מקפיאים אותה לחלוטין**, ומוסיפים שתי מטריצות קטנות A ו- B בממדים נמוכים. מאמנים רק את שתי המטריצות הקטנות, במקומות הרלוונטיים בלבד:

$$W = W_0 + BA$$

כך מתקבל חיסכון של סדרי גודל במספר הפרמטרים הניתנים לאימון.

דוגמה מהשטח

סטודנט רצה לאמן מודל Text-to-Image לסגנון אמנות יפנית בשחור-לבן. עם כ-900 תמונות בלבד ובאמצעות LoRA, הוא בנה מודל מצוין שעובד נהדר. האימון התבצע על מחשב עם כרטיס בן 16GB (בעלות של כ-6,000 ש"ח) ונמשך כחצי שעה בלבד.

שלד אימון LoRA עם PEFT

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM

base = AutoModelForCausalLM.from_pretrained("meta-llama/Llama
-3-8B")

config = LoraConfig(
    r=8,                                     # rank of A and B matrices
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
)

model = get_peft_model(base, config)
model.print_trainable_parameters()    # only A,B are trainable
```


; W_0 is frozen

7.4 QLoRA ו-OLoRA

QLoRA משלב קוונטיזציה עם LoRA: לא רק חיסכון במטריצות, אלא גם הורדת הדיוק של המטריצות עצמן, מה שמאפשר לקחת מודלים רגילים ולהריץ אותם בכוח חישוב סביר.

OLoRA: ב-LoRA רגיל האתחול של המטריצות A ו- B אקראי, ולכן האימון עלול להיות לא יציב בתחילתו. OLoRA משתמש באתחול אורתונורמלי באמצעות פירוק QR (כל מטריצה ניתנת לתיאור כמכפלה של שתי מטריצות), והתוצאה היא אימון חלק ויציב יותר.

7.5 הרחבה – הבסיס המתמטי של LoRA ו-OLoRA

שיטת LoRA הוצגה על ידי חוקרי Microsoft (Hu et al., ICLR 2022). בהינתן מטריצת משקלים מקורית W_0 בגודל $d \times k$, השיטה מקפיאה אותה ומוסיפה מסלול למידה מקביל דרך שתי מטריצות קטנות: B בגודל $d \times r$ ו- A בגודל $r \times k$, כאשר דרגת ההתאמה נמוכה בהרבה מן הממדים המקוריים, $r \ll \min(d, k)$. החישוב הוא $W = W_0 + BA$. בתום האימון המטריצות מוכפלות וממוזגות חזרה אל המשקלים, ולכן אין כל פגיעה בזמן ההסקה (Inference Latency).

OLoRA (Büyükkakyüz, 2024) משפר את שלב האתחול: במקום ערכים אקראיים, מבוצע פירוק QR של מטריצת המשקלים, $W = QR$ (כאשר Q אורתוגונלית ו- R משולשית עליונה), והמטריצות מאותחלות מתוך מרחבים אלו. אתחול אורתונורמלי זה מבטיח שהלמידה תתרחש מלכתחילה בתת-מרחב מותנה-היטב (well-conditioned), ומכאן ההתכנסות המהירה והיציבה. מאחר שפירוק ה-QR מתבצע פעם אחת בלבד בשלב האתחול, עלותו מתקזזת (amortized) והופכת לזניחה לאורך עשרות אלפי מחזורי האימון.

8 AirLLM: הרצת מודלים ענקיים על CPU

8.1 הרעיון

AirLLM הוא פרויקט קוד פתוח המיועד בעיקר למחקר, המאפשר להריץ מודלים כבדים בתוך CPU (וגם GPU). במקום לטעון את כל המשקלים ל-VRAM בבת אחת, AirLLM עובד **שכבה אחר שכבה**: מעלה שכבה, מבצע, מקבל תשובה, מביא את השכבה הבאה וכן הלאה. הרעיון מבוסס על **זיכרון וירטואלי** ו-Paging ממערכות הפעלה (שכבה = page).

8.2 תזכורת: זיכרון וירטואלי ממערכות הפעלה

הזיכרון הווירטואלי גורם למחשב "לחשוב" שיש לו זיכרון גדול בהרבה מהפיזי (למשל 1TB במקום 32GB), בעוד בכל רגע מובאת רק המידע הרלוונטי. עקרונות המפתח:

- **Page Table** ו-**Pages** – הזיכרון הווירטואלי מחולק לדפים בגודל קבוע; טבלת הדפים מתרגמת בין דף וירטואלי למסגרת פיזית ב-RAM.
- **MMU** – היחידה המבצעת את התרגום בזמן אמת.
- **Miss / Hit** – אם הדף קיים ב-RAM זו פגיעה (hit); אחרת יש להביאו מהדיסק ולפנות דף אחר (miss).
- **היררכיית זיכרון** – ככל שהזיכרון קרוב למעבד הוא מהיר ויקר יותר: Registers → Cache → RAM → NVMe/SSD.
- **עקרון הלוקליות** – הסתברות גבוהה שהפעולה הבאה תהיה סמוכה לנוכחית (שורה 17 ואז 18); על כך מבוסס ה-Cache.

- **mmap** – מאפשר להתייחס לקובץ כאילו היה אזור בזיכרון; מערכת ההפעלה טוענת דפים רק כשניגשים אליהם.

AirLLM מיישם היגיון דומה עבור מודלי שפה גדולים: לא כל המידע חייב להיות ב-RAM בזמן.

8.3 המחיר: Latency מול זיכרון

צוואר הבקבוק האמיתי הוא ה-I/O – הזמן להביא ולפנות דף. לכן AirLLM מאפשר זיכרון קטן בהרבה, אך במחיר של latency גבוה (ירידה בתפוקה ועלייה בזמן התגובה). הוא מתאים להתנסות, מחקר ונגישות – פחות למערכות זמן-אמת עמוסות. בלי לשלם טוקנים וכוח חישוב יקר, אפשר לבצע דברים "במחיר של חשמל" ובמחיר של המתנה.

8.4 הרחבה – כיצד AirLLM מנצל את mmap

המימוש בפועל של AirLLM נשען על mmap בשילוב עם פורמט ה-SafeTensors השטוח. בקריאת mmap מערכת ההפעלה יוצרת מיפוי לוגי בלבד במרחב הכתובות הווירטואלי, ללא טעינת ביטים בפועל. כאשר המנוע ניגש לשכבה כדי לחשבה (Forward Pass), נוצרת חריגת דף (Page Fault); הליבה (Kernel) מעלה את הבלוק הנדרש למטמון הדפים שלה (Page Cache) ללא העתקות מיותרות למרחב המשתמש (Zero-copy).

לאחר החישוב, ה-GPU שומר ב-VRAM רק את מצבי הביניים הקטנים (Hidden States), ומשקלי השכבה הקודמת מתפנים אוטומטית לטובת השכבה הבאה הנטענת באותה טכניקה – בדיוק כפי שעובד עקרון ה-Paging במערכות הפעלה.

8.5 ידע מתקדם – חזית המחקר: LLM in a Flash, FlexGen ו-PagedAttention

האנלוגיה למערכות הפעלה עומדת בבסיסם של מחקרים מובילים נוספים, המעידים עד כמה החומר נמצא בחזית הידע (State-of-the-Art):

- **FlexGen** (ICML 2023) – ביסס מודל מתמטי (Linear Programming) לתזמון אופטימלי של פינוי וטעינה בין ה-GPU, הזיכרון והדיסק, ואיפשר להריץ מודל של 175 מיליארד פרמטר על כרטיס ביתי יחיד בן 16GB.

- **LLM in a Flash** (Apple, 2024) – ייעל טעינה מדורגת מדיסק Flash על ידי חיזוי אילו נוירונים יתאפסו עקב פונקציית ה-ReLU (Sparsity), וכך קרא רק את עמודות הזיכרון הרלוונטיות.
- **PagedAttention** (SOSP 2023) – הבסיס לספריית vLLM; מחיל את עקרון ה-Paging וטבלאות הדפים ישירות על ה-KV Cache שב-VRAM. כך בוטל הקיטוע (Fragmentation) שבזבז עד 60% מהזיכרון, התאפשר שיתוף דפים בין בקשות עם פרומפט משותף, והתפוקה הכוללת עלתה פי 2 עד 4.

9 המשימה המעשית

9.1 מה צריך לעשות

המטרה היא להוכיח ש-AirLLM עוזר במקרים מסוימים. שלבי המשימה:

1. לבחור משימה ולבחור מודל ב-Hugging Face, תוך התאמה למפרט המחשב.
2. להתקין את Ollama ואת המודל ולהריץ הרצה בסיסית כדי לוודא שהכול עובד.
3. לקחת מודל **גדול מדי** שאינו נכנס ל-RAM/GPU ולהראות שהוא נכשל או איטי להחריד (baseline).
4. להריץ את אותו מודל באמצעות AirLLM על CPU ולהראות שהפעם הוא **כן** רץ – במחיר של latency.
5. **למדוד**: זמן תגובה, צריכת זיכרון וזמן ריצה, ולהשוות בין CPU, GPU ו-AirLLM.

טיפים והמלצות

- ליצור סביבה וירטואלית (venv); מומלץ עם uv.
- **לא** לעבוד עם גרסת Python החדשה ביותר – חבילות רבות עדיין אינן מותאמות לה.
- לאבטח את ה-Token של Hugging Face: לא לשמור אותו גלוי.
- להתחיל ממודל קטן ולא רלוונטי, ולהגדיר מספר טוקנים מרבי נמוך, רק כדי לבדוק שהצנרת עובדת.
- להקצות מספיק מקום בדיסק לפני הורדת מודלים גדולים.

9.2 סיכום המסע

עברנו מסע שלם: מהחומרה (GPU/CPU, SASS/PTX/CUDA), דרך שלבי ההסקה (Decode/Prefill) ו-VRAM, דרך בחירת מודלים ב-Hugging Face, רישוי, פורמטים (GGUF/SafeTensors) וקוונטיזציה, אל הרצה מקומית ב-Ollama, אימון יעיל ב-OLoRA/QLoRA/LoRA, ולבסוף הרצת מודלים ענקיים על CPU בעזרת AirLLM ועקרונות הזיכרון הווירטואלי. כל אלה יחד מרחיבים את הגבול של מה שאפשר להריץ ולאמן באופן מקומי.